System 1: Template-Based Generator

1. Overview

For the first system, I built a template-based text generator using the E2E dataset. The idea is simple: look at how the training references describe restaurants, turn those into templates by replacing specific details with placeholders, and then at test time fill those placeholders back in with the actual values from the meaning representation (MR). This approach follows the style described in Smiley et al. (2018), but I kept it fairly straightforward so that the system is reliable and easy to run.

2. System Design and Implementation

2.1 Data Preparation

The dataset contained pairs of MRs and reference sentences. Each MR encodes attributes such as name, eatType, food, and area. Before extracting templates, I parsed each MR into attribute–value pairs and ignored the "familyFriendly" attribute as required by the assignment. On the text side, the reference sentences were processed through delexicalization: each attribute value was replaced with its attribute name in uppercase.

Regular expressions were used for these replacements because they are simple and effective, and sorting values by length ensured that longer phrases were substituted before shorter ones. This step gave me generalized templates that could later be re-lexicalized with new values at test time.

2.2 Template Creation

Once the reference texts were delexicalized, they were grouped by their attribute sets. For each group, I kept only the most frequent template. The motivation here was to reduce noise and ensure that the system favored the dominant, more natural phrasing found in the training data, rather than relying on rare or unusual sentences.

2.3 Text Generation

At test time, the system tries three strategies. First, it looks for an exact attribute match and re-lexicalizes the corresponding template. If no exact match exists, it searches for the largest subset of attributes with a template and uses that instead. This allows the model to generalize to unseen attribute combinations. If no template can be found at all, the system falls back to a very simple default sentence such as *"Alimentum is a restaurant."* This layered design makes the system robust: it can handle exact matches, unseen combinations, and even completely new cases without breaking.

3. Reflections and Conclusion

When I designed this system, I tried to balance simplicity with reliability. Using regular expressions to replace attribute values turned out to be a practical choice because it avoids mistakes when words overlap, and it makes the delexicalization step more consistent. I also decided to keep only the most common template for each attribute set. This reduces noise and prevents the system from producing rare or awkward phrasing that only shows up once or twice in the training data. Another important feature was the use of partial templates, which allows the system to generalize to new attribute combinations it hasn't seen before, while the fallback rule makes sure that it always produces at least one valid sentence instead of failing silently. Of course, the trade-off is that the output often lacks variety, since the same template is reused whenever the same attributes appear, and the system cannot adapt when the training data described attributes in a different way. Still, the method provides a dependable baseline that captures the main information and produces fluent text most of the time.

Overall, this first system shows how far you can get with just templates and some simple pattern matching. It may not be as flexible or creative as a neural network, but it is fast, consistent, and provides a strong baseline to compare against more advanced models.

System 2: Neural Network Generator

1. Introduction

The second system is a neural-based text generator using a simple Transformer model. The goal was to see how well a standard encoder–decoder network with self-attention could learn to produce descriptions from meaning representations (MRs). Compared to the template system, I expected this model to capture more variation in the data and generate more flexible outputs.

2. Data Preparation

The dataset was processed by sampling 10% of the training set to keep training manageable on limited hardware. Both the MRs and references were tokenized, and a vocabulary of around 2,500 words was built by keeping tokens that appeared at least twice. Four special tokens were added: <PAD>, <SOS>, <EOS>, and <UNK>. This setup allowed the model to represent sentences consistently while handling unknown or rare words.

3. Model Design and Implementation

3.1 Word and MR Representation

The MRs were treated as simple token sequences rather than structured inputs. Each word was mapped into an embedding vector of size 128, with added positional embeddings so the model could distinguish token order. References were prepared with <SOS> at the start and <EOS> at the end so that the decoder could learn to generate sequences step by step.

3.2 Network Architecture

The architecture was based on the standard Transformer encoder–decoder design provided by PyTorch. The encoder processed the MR embeddings into hidden representations, and the decoder generated text autoregressively. I used two layers in both encoder and decoder, multi-head attention with 4 heads, and a feed-forward size of 512 with dropout 0.25. The final layer was a fully connected projection to vocabulary size.

This setup was deliberately kept lightweight because of hardware constraints. I also tried deeper models and adjusted the number of attention heads, but these changes did not lead to higher BLEU scores.

3.3 Loss Function and Optimizer

The training objective was cross-entropy loss with <PAD> tokens ignored. The optimizer was Adam with a learning rate of 0.0006. Gradient clipping (at 1.0) was applied to prevent exploding gradients, which occasionally happened during early experiments.

3.4 Text Generation

During testing, the model generated outputs with a small beam search (beam width = 3). This was added to reduce random errors and hallucinations by exploring multiple possible sequences instead of greedy decoding. Generated sequences stopped once an <EOS> token was produced or the maximum length of 25 was reached.

4. Evaluation

The system was evaluated on testset_w_refs.csv using BLEU. The implementation followed the same procedure as the template system: references were compared against generated outputs, and the BLEU score was calculated with smoothing.

The results showed that the BLEU score of the neural model (0.09) was noticeably lower than that of the template-based system (0.28). Even after experimenting with multiple hyperparameters, adding extra layers, and tuning dropout or learning rate, the performance did not improve significantly. In particular, the use of self-attention did not lead to the expected performance gains. While Transformers are powerful for large datasets, in this setting they struggled with limited data and short training.

5. Discussion and Conclusion

This outcome highlights an important point: more advanced models do not always guarantee better performance, especially when resources are limited. The template system had the advantage of being tightly aligned with the training references, so its outputs matched test references more closely, leading to higher BLEU. By contrast, the neural model needed more data and training time to capture language patterns effectively, which explains why its BLEU score was noticeably lower.

That said, I also printed a few outputs from the neural system alongside their references during testing. On inspection, I found that the model was able to generate grammatical sentences that conveyed the main information from the MR, even if the wording did not always match the reference closely enough to score well under BLEU. In this sense, the Transformer produced text that was often reasonable and fluent, and while the score suggests weaker performance, the qualitative results showed that it was still a fair trade-off between variation and accuracy. This also reinforces the idea that BLEU alone does not fully capture the quality of generation.